

Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики

Звіт

з лабораторної роботи No1
з моделювання складних систем

Варіант 1

Виконала:
Студентка групи ІПС-32
Клевчук Марія Вячеславівна

Київ
2025

Тема: Побудова лінійної моделі з допомогою псевдообернених операторів.

Постановка задачі:

Вважаємо, що на вхід системи перетворення, математична модель якої невідома, подається m -ї розмірності вектор x_j , він є вхідним сигналом. На виході отримуємо вектор y_j розмірності p . Маємо знайти оператор P перетворення вихідного сигналу на вхідний.

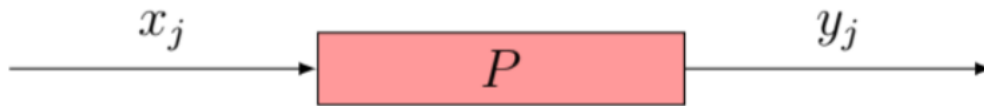


Рис. 3.1: Математична модель

Для послідовності вхідних сигналів $x_j, j = 1, 2, \dots, n$ та вихідних сигналів $y_j, j = 1, 2, \dots, n$ будемо шукати математичну модель оператора об'єкту в класі лінійних операторів.

$$A \begin{pmatrix} \mathbf{x}_j \\ 1 \end{pmatrix} = \mathbf{y}_j, \quad j = 1, 2, \dots, n.$$

Цю систему можемо записати у матричній формі

$$A \begin{pmatrix} x_1 & x_2 & \dots & x_n \\ 1 & 1 & \dots & 1 \end{pmatrix} = (y_1, y_2, \dots, y_n)$$

або $AX=Y$,

де $X = \begin{pmatrix} x_1 & x_2 & \dots & x_n \\ 1 & 1 & \dots & 1 \end{pmatrix}$ – матриця вхідних сигналів розмірності $m \times n$, $Y = (y_1, y_2, \dots, y_n)$ – матриця вихідних сигналів розмірності $p \times n$.

Тоді $A = YX^+ + VZ^T(X^T)$, де матриця V - довільна (наприклад нульова) матриця $p \times m$, $Z(X^T) = I_m - XX^+$.

Для пошуку псевдообернених матриць будемо використовувати два методи: алгоритм, заснований на означенні Мура-Пенроуза та алгоритм на основі формули Гревіля.

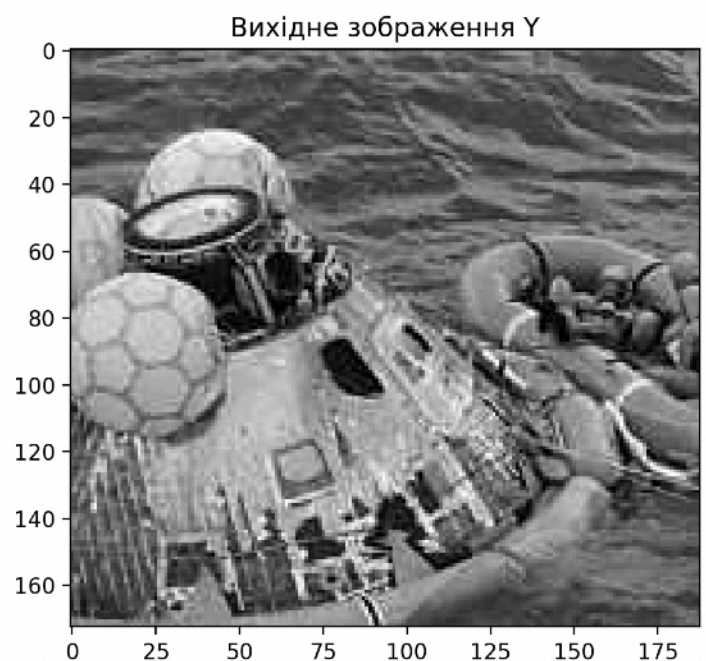
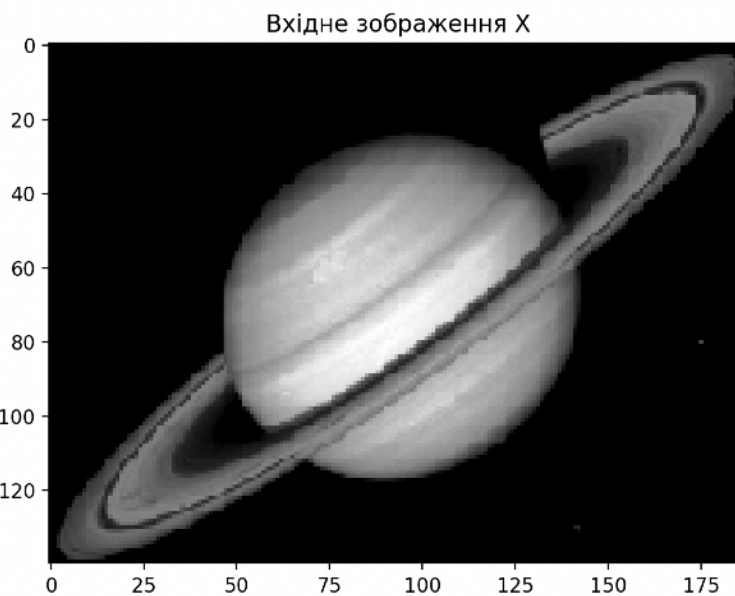
Хід роботи:

Зчитуємо вхідний сигнал $x1$ та вихідний сигнал $y1$, та виводимо зображення.

```
# Зчитування вхідного та вихідного зображення
X = np.array(io.imread("x1.bmp"), dtype=np.float64)
Y = np.array(io.imread("y1.bmp"), dtype=np.float64)

# Відображення вхідних і вихідних даних
show_image(X, title: "Вхідне зображення X")
show_image(Y, title: "Вихідне зображення Y")
```

Результат:



Реалізуємо перевірку правильності знаходження псевдооберненої матриці за теоремою:

Теорема 3.1 (характеристична властивість псевдооберненої матриці).
Матриця A^+ розмірності $n \times t$ є псевдооберненою матрицею до матриці A розмірності $t \times n$ тоді і тільки тоді, якщо виконуються такі умови:

- $AA^+A = A$;
- $A^+AA^+ = A^+$;
- AA^+ – симетрична матриця розмірності $t \times t$;
- A^+A – симетрична матриця розмірності $n \times n$.

Реалізація в коді:

```
# Функція для перевірки правильності знаходження псевдооберненої матриці
def check_pinv_properties(A, A_plus): 2 usages
    m, n = A.shape
    # Перевірка розмірів добутоків
    AA_plus = A @ A_plus
    A_plusA = A_plus @ A
    if AA_plus.shape != (m, m): return False
    if A_plusA.shape != (n, n): return False
    # Умови Мура-Пенроуза
    if not np.allclose(AA_plus @ A, A): return False
    if not np.allclose(A_plusA @ A_plus, A_plus): return False
    if not np.allclose(AA_plus, AA_plus.T): return False
    if not np.allclose(A_plusA, A_plusA.T): return False
    return True
```

Перейдемо до реалізації алгоритму пошуку псевдооберненої матриці методом Гревіля.

Цей алгоритм є рекурентим. Його суть полягає в послідовному додаванні рядків до матриці A і використанні формули Гревіля для обчислення псевдооберненої матриці до розширеної матриці на кожному кроці, виходячи з псевдооберненої матриці попереднього кроку. Представляємо матрицю A у вигляді,

$$A = \begin{pmatrix} a_1^T \\ a_2^T \\ \vdots \\ a_n^T \end{pmatrix}.$$

де a_i^T — i -тий рядок матриці A .

Формуля Гревіля має вигляд

$$\begin{pmatrix} A \\ a^T \end{pmatrix}^+ = \begin{cases} \left(A^+ - \frac{Z(A)aa^TA^+}{a^TZ(A)a} : \frac{Z(A)a}{a^TZ(A)a} \right), & \text{if } a^TZ(A)a > 0 \\ \left(A^+ - \frac{R(A)aa^TA^+}{1+a^TR(A)a} : \frac{R(A)a}{1+a^TR(A)a} \right), & \text{if } a^TZ(A)a = 0 \end{cases}, \quad (3.2)$$

де $Z(A) = E - A^+A$ — проектор на ядро матриці A , $R(A) = A^+(A^+)^T$.

A — матриця з попереднього кроку, a^T — новий рядок, що додається, A^+ — псевдообернена матриця для A .

Для першого кроку $(a_1^T)^+ = \frac{a_1}{a_1^T a_1}$, при $a_1 \neq 0$; при $a_1 = 0$ $(a_1^T)^+ = 0$,

На наступному кроці додаємо рядок (a_2^T), шукаємо псевдообернену матрицю для

$$\begin{pmatrix} a_1^T \\ a_2^T \end{pmatrix}$$

Продовжуємо алгоритм доки не будуть вичерпані всі рядки матриці A .

Реалізація в коді:

```
def greville(A):  
    """usage  
    m, n = A.shape  
    # Крок 1: Обробка першого рядка a_1^T  
    a1 = A[0, :].reshape(-1, 1)  
    denom = a1.T @ a1  
    if denom[0, 0] != 0:  
        A_plus_i = a1 / denom[0, 0]  
    else:  
        A_plus_i = np.zeros((n, 1))  
    A_i = A[0:1, :]  
  
    # Цикл з додаванням рядків  
    for i in range(1, m):  
        ai_T = A[i:i + 1, :]  
        Z_A = np.eye(n) - A_plus_i @ A_i  
        ai = ai_T.T  
        # Умова для формули Гревілья  
        condition = ai.T @ Z_A @ ai  
        # Оновлення A_plus  
        if condition[0, 0] > 1e-10:  
            v = (Z_A @ ai) / condition[0, 0]  
  
            term = (Z_A @ ai @ ai.T @ A_plus_i) / condition[0, 0]  
            A_plus_next = np.hstack([A_plus_i - term, v])  
        else:  
            R_A = A_plus_i @ A_plus_i.T  
            den = 1 + ai.T @ R_A @ ai  
            v = (R_A @ ai) / den[0, 0]  
  
            term = (R_A @ ai @ ai.T @ A_plus_i) / den[0, 0]  
            A_plus_next = np.hstack([A_plus_i - term, v])  
        # Оновлення для наступної ітерації  
        A_plus_i = A_plus_next  
        A_i = np.vstack([A_i, ai_T])  
  
    return A_plus_i
```

Викликаємо функцію, та після її виконання перевіряємо правильність знаходження матриці за теоремою:

```
# Виконуємо алгоритм Гревілья
X_grev_plus = greville(X)
if check_pinv_properties(X, X_grev_plus):
    print("Гревіль: псевдообернена матриця коректна.")
```

У консолі отримали повідомлення, що алгоритм спрацював коректно:

```
Гревіль: псевдообернена матриця коректна.
```

Тепер знайдемо A за відомою формулою $A = YX^+ + VZ^T(X^T)$. Знайдемо і перетворений сигнал $Y = AX$ та виведемо результат:

```
V = np.zeros((p, m))
Z = np.eye(m) - X @ X_grev_plus
A_grev = Y @ X_grev_plus + V @ Z.T

Y_grev = np.clip(A_grev @ X, a_min: 0, a_max: 255)
show_image(Y_grev, title: "Результат за Гревілем")
```



Перейдемо до реалізації алгоритму, заснованого на означенні Мура-Пенроуза. Цей алгоритм наближено шукає псевдообернену матрицю. Він використовує граничний перехід, який присутній в означенні, щоб сформувати числовий ітераційний процес.

Означення псевдооберненої матриці A^+ до матриці A розмірності $m \times n$ ґрунтується на граничному переході:

$$A^+ = \lim_{\delta^2 \rightarrow 0} \left\{ (A^T A + \delta^2 E_n)^{-1} A^T \right\} = \lim_{\delta^2 \rightarrow 0} \left\{ A^T (A A^T + \delta^2 E_m)^{-1} \right\}.$$

Будемо використовувати наближену формулу

$$A^+ \approx A^T (A A^T + \delta_0^2 E_m)^{-1}.$$

Значення δ підбирається експериментально. Оскільки воно має бути близьким до нуля, застосовується ітераційна схема для його зменшення та підвищення точності наближення:

- Задається початкове наближення $\delta = \delta_0$ та критерій зупинки ε .
- Розраховується початкове наближення:

$$A_0^+ = A^T (A A^T + \delta_0^2 E_m)^{-1};$$

- На кожному наступному кроці k значення δ зменшується вдвічі:
- На кожному наступному кроці k рахується наближення псевдооберненої матриці:

$$\delta_k = \frac{\delta_{k-1}}{2};$$

$$A_k^+ = A^T (A A^T + \delta_k^2 E_m)^{-1}$$

- Процес зупиняється за умови, якщо норма різниці між поточним та попереднім наближенням менша за задану точність:

$$\| A_k^+ - A_{k-1}^+ \| < \varepsilon;$$

Тоді шукана псевдообернена матриця $A^+ = A_k^+$.

Реалізація в коді:

```
# Алгоритм за означенням Мура-Пенроуза
def moore_penrose(A, eps=1e-8): 1 usage
    m, _ = A.shape
    delta = 10.0
    A_prev = A.T @ np.linalg.inv(A @ A.T + (delta**2) * np.eye(m))

    while True:
        delta /= 2
        A_cur = A.T @ np.linalg.inv(A @ A.T + (delta**2) * np.eye(m))
        if np.linalg.norm(A_cur - A_prev) < eps:
            return A_cur
        A_prev = A_cur
```

Викличемо функцію та перевіримо коректність її виконання за теоремою. З виводу в консоль розуміємо, що програма спрацювала коректно:

```
# Виконуємо алгоритм за означенням Мура-Пенроуза
X_mp_plus = moore_penrose(X)
if check_pinv_properties(X, X_mp_plus):
    print("Мур-Пенроуз: псевдообернена матриця коректна.")
```

Мур-Пенроуз: псевдообернена матриця коректна.

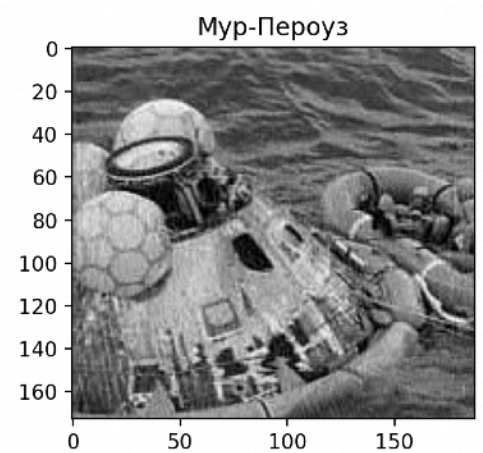
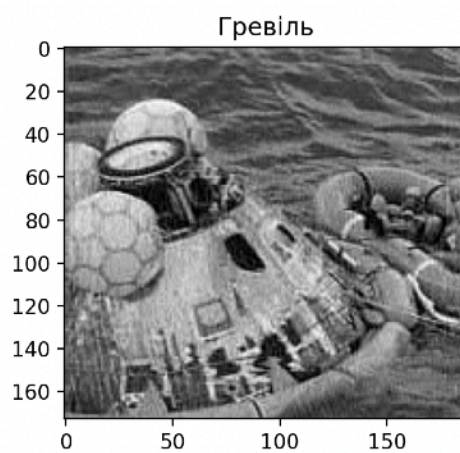
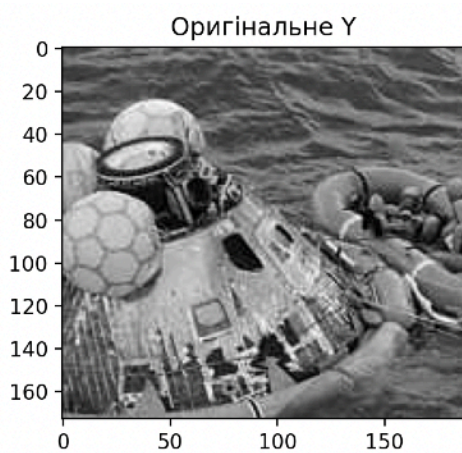
Тепер знову порахуємо матрицю A та новий вихідний сигнал Y . Виведемо зображення:

```
Z = np.eye(m) - X @ X_mp_plus
A_mp = Y @ X_mp_plus + V @ Z.T

Y_mp = np.clip(A_mp @ X, a_min: 0, a_max: 255)
show_image(Y_mp, title: "Результат за Муром-Пенроузом")
```



Виведемо всі три вихідні зображення: початкове, отримане з допомогою алгоритму Гревіль та алгоритму за означенням Мура-Пенроуза. Зображення дуже близькі між собою.



```

import numpy as np
import matplotlib.pyplot as plt
import imageio.v3 as iio

# Функція для виведення зображення
def show_image(matrix, title):
    plt.figure()
    plt.imshow(matrix.astype(np.uint8), cmap='gray')
    plt.title(title)
    plt.show(block=True)

# Функція для перевірки правильності знаходження псевдооберненої
# матриці
def check_pinv_properties(A, A_plus):
    m, n = A.shape
    # Перевірка розмірів добутків
    AA_plus = A @ A_plus
    A_plusA = A_plus @ A
    if AA_plus.shape != (m, m): return False
    if A_plusA.shape != (n, n): return False
    # Умови Мура–Пенроуза
    if not np.allclose(AA_plus @ A, A): return False
    if not np.allclose(A_plusA @ A_plus, A_plus): return False
    if not np.allclose(AA_plus, AA_plus.T): return False
    if not np.allclose(A_plusA, A_plusA.T): return False
    return True

# Алгоритм Гревіля
def greville(A):
    m, n = A.shape
    # Крок 1: Обробка першого рядка  $a_1^T$ 
    a1 = A[0, :].reshape(-1, 1)
    denom = a1.T @ a1

```

```

if denom[0, 0] != 0:
    A_plus_i = a1 / denom[0, 0]
else:
    A_plus_i = np.zeros((n, 1))
A_i = A[0:1, :]
# Цикл з додаванням рядків
for i in range(1, m):
    ai_T = A[i:i + 1, :]
    Z_A = np.eye(n) - A_plus_i @ A_i
    ai = ai_T.T
    # Умова для формули Гревіля
    condition = ai.T @ Z_A @ ai
    # Оновлення A_plus
    if condition[0, 0] > 1e-10:
        v = (Z_A @ ai) / condition[0, 0]
        term = (Z_A @ ai @ ai.T @ A_plus_i) / condition[0, 0]
        A_plus_next = np.hstack([A_plus_i - term, v])
    else:
        R_A = A_plus_i @ A_plus_i.T
        den = 1 + ai.T @ R_A @ ai
        v = (R_A @ ai) / den[0, 0]
        term = (R_A @ ai @ ai.T @ A_plus_i) / den[0, 0]
        A_plus_next = np.hstack([A_plus_i - term, v])
    # Оновлення для наступної ітерації
    A_plus_i = A_plus_next
    A_i = np.vstack([A_i, ai_T])
return A_plus_i
# Алгоритм за означенням Мура-Пенроуза
def moore_penrose(A, eps=1e-8):

```

```

m, _ = A.shape

delta = 10.0

A_prev = A.T @ np.linalg.inv(A @ A.T + (delta**2) * np.eye(m))

while True:

    delta /= 2

    A_cur = A.T @ np.linalg.inv(A @ A.T + (delta**2) *
np.eye(m))

    if np.linalg.norm(A_cur - A_prev) < eps:

        return A_cur

    A_prev = A_cur

# Зчитування вхідного та вихідного зображення
X = np.array(io.imread("x1.bmp"), dtype=np.float64)
Y = np.array(io.imread("y1.bmp"), dtype=np.float64)

# Відображення вхідних і вихідних даних
show_image(X, "Вхідне зображення x")
show_image(Y, "Вихідне зображення y")

# Додаємо рядок одиниць (зсув)
X = np.append(X, np.ones((1, X.shape[1])), axis=0)

m, n = X.shape
p = Y.shape[0]

# Виконуємо алгоритм Гревілья
X_grev_plus = greville(X)

if check_pinv_properties(X, X_grev_plus):

    print("Гревіль: псевдообернена матриця коректна.")

V = np.zeros((p, m))

Z = np.eye(m) - X @ X_grev_plus

A_grev = Y @ X_grev_plus + V @ Z.T

Y_grev = np.clip(A_grev @ X, 0, 255)

show_image(Y_grev, "Результат за Гревілем")

```

```

# Виконуємо алгоритм за означенням Мура-Пенроуза
X_mp_plus = moore_penrose(X)
if check_pinv_properties(X, X_mp_plus):
    print("Мур-Пенроуз: псевдообернена матриця коректна.")
Z = np.eye(m) - X @ X_mp_plus
A_mp = Y @ X_mp_plus + V @ Z.T
Y_mp = np.clip(A_mp @ X, 0, 255)
show_image(Y_mp, "Результат за Муром-Пенроузом")
# Порівняння результатів
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(Y.astype(np.uint8), cmap='gray')
plt.title("Оригінальне Y")
plt.subplot(1, 3, 2)
plt.imshow(Y_grev.astype(np.uint8), cmap='gray')
plt.title("Гревіль")
plt.subplot(1, 3, 3)
plt.imshow(Y_mp.astype(np.uint8), cmap='gray')
plt.title("Мур-Пероуз")
plt.show()

```

Виведення:

```

Гревіль: псевдообернена матриця коректна.
Мур-Пенроуз: псевдообернена матриця коректна.

```